

# 20 pandas Interview Questions, Free Sample

pandas

Every question was generated from real code, run for real, and checked against the actual output before being kept. Every wrong answer is explained too, not just the right one.

## How this book was built

Most interview question banks are written by hand and quietly carry the mistakes of whoever wrote them. This one was built the other way around. Every code snippet in this book was run for real, and the printed answer key is the printed output, not someone's memory of what it should be. Where two independent ways of running the same snippet disagreed, the question was dropped rather than guessed at.

Each question also explains why the three wrong answers are wrong, not only why the right one is right. That is the part most practice tests skip, and it is the part that actually teaches you something the second time you get a question wrong.

# pandas Questions

## Block 1, Series, Index and dtypes

1. What does this print?

```
import pandas as pd
s = pd.Series([1, True])
print(s.tolist())
```

- A. [1, 1]
- B. [1, True]
- C. [True, True]
- D. [1.0, 1.0]

2. What does this print?

```
import pandas as pd
s = pd.Series(['1', '2', 'x'])
print(s.astype(int))
```

- A. ValueError
- B. nan
- C. 0 1\n1 2\n2 NaN\ndtype: float64
- D. 0 1\n1 2\n2 0\ndtype: int64

3. What does this print?

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
print(s[s > 3].tolist())
```

- A. [4, 5]
- B. [3, 4, 5]
- C. [True, True]
- D. [1, 2, 3]

## Block 2, Selecting with loc, iloc and Masks

4. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2, 3, 4], 'b': [5, 6, 7, 8]})
print(df[df['a'].isin([1, 4])]['b'].tolist())
```

- A. [5, 8]
- B. [6, 7]
- C. [5, 6, 7, 8]
- D. []

5. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]}, index=[10, 20, 30])
print(df.iloc[-1]['a'])
```

- A. 10
- B. 2
- C. 3
- D. 1

## Block 3, Missing Data, NaN and NA

6. What does this print?

```
import pandas as pd
s = pd.Series([1, 2, None])
print(s.sum())
```

- A. 2.0
- B. 3
- C. 3.0
- D. nan

7. What does this print?

```
import pandas as pd
s = pd.Series([1, None, 3])
s2 = s.fillna(0)
print(s.tolist())
```

- A. [1.0, nan, 3.0]
- B. [1, None, 3]
- C. [1.0, 0.0, 3.0]
- D. [1.0, 3.0]

8. What does this print?

```
import pandas as pd
a = pd.array([1, pd.NA, 3], dtype='Int64')
b = pd.array([1, 2, pd.NA], dtype='Int64')
print((a + b).tolist())
```

- A. [<NA>, <NA>, <NA>]
- B. [2, <NA>, <NA>]
- C. [2, 2, <NA>]
- D. [2, 2, 3]

## Block 4, GroupBy and Aggregation

9. What does this print?

```
import pandas as pd
df = pd.DataFrame({'k': ['b', 'a', 'b', 'a'], 'v': [1, 2, 3, 4]})
print(df.groupby('k')['v'].sum().index.tolist())
```

- A. ['b', 'a']
- B. [0, 1]
- C. ['a', 'b']
- D. ['a', 'a', 'b', 'b']

10. What does this print?

```
import pandas as pd
df = pd.DataFrame({'k': ['a', 'a', 'b'], 'v': [1, 2, 3]})
print(df.groupby('k')['v'].agg(lambda s: (s ** 2).sum()).to_dict())
```

- A. {'a': 5, 'b': 3}
- B. {'a': 3, 'b': 3}
- C. {'a': 5, 'b': 9}
- D. {'a': 9, 'b': 9}

## Block 5, Merge, Join and Concat

11. What does this print?

```
import pandas as pd
left = pd.DataFrame({'k': [1, 1, 2], 'a': ['a1', 'a2', 'a3']})
right = pd.DataFrame({'k': [1, 1, 3], 'b': ['b1', 'b2', 'b3']})
merged = pd.merge(left, right, on='k')
print(merged.shape)
```

- A. (3, 3)
- B. (6, 3)
- C. (4, 3)
- D. (2, 3)

12. What does this print?

```
import pandas as pd
a = pd.DataFrame({'x': [1, 2]})
b = pd.DataFrame({'y': [3, 4]})
combined = pd.concat([a, b], axis=1)
print(list(combined.columns))
```

- A. ['x\_x', 'y\_y']
- B. ['x', 'x']
- C. ['y', 'x']
- D. ['x', 'y']

13. What does this print?

```
import pandas as pd
s1 = pd.Series([1, 2], name='s')
s2 = pd.Series([3, 4], name='s')
combined = pd.concat([s1, s2])
print(type(combined).__name__, combined.tolist())
```

- A. list [1, 2, 3, 4]
- B. Series [1, 2]

- C. DataFrame [1, 2, 3, 4]
- D. Series [1, 2, 3, 4]

## Block 6, Reshaping, Pivot and Melt

14. What does this print?

```
import pandas as pd
df = pd.DataFrame({'id': [1, 2], 'a': [10, 20]})
melted = df.melt(id_vars='id')
print(list(melted.columns))
```

- A. ['id', 'variable', 'value']
- B. ['id', 'a', 'variable', 'value']
- C. ['variable', 'value']
- D. ['id', 'value', 'variable']

15. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 1, 2], 'b': [3, 1, 2], 'orig': [0, 1, 2]})
sorted_df = df.sort_values('a')
print(sorted_df.shape)
```

- A. (1, 3)
- B. (3, 2)
- C. (3, 3)
- D. (2, 3)

## Block 7, Copy-on-Write, Views and Assignment

16. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2, 3], 'b': [7, 8, 9]})
df.loc[df['a'] > 1, 'b'] = 0
print(df['b'].tolist())
```

- A. ChainedAssignmentError
- B. [7, 8, 9]
- C. [7, 0, 0]
- D. [0, 0, 0]

17. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 1, 2]})
resultat = df.drop_duplicates(inplace=True)
print(resultat, df['a'].tolist())
```

- A. None [1, 2]
- B. None [1, 1, 2]
- C. a\n0 1\n2 2 [1, 2]

D. [1, 2] [1, 2]

18. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2, 3]})
df.loc[:, 'a'] = df['a'] * 10
print(df['a'].tolist())
```

- A. [10, 10, 10]
- B. None
- C. [10, 20, 30]
- D. [1, 2, 3]

## Block 8, Strings, Dates and apply

19. What does this print?

```
import pandas as pd
s = pd.Series(['apple', 'banana', 'avocado'])
print(s.str.startswith('a').tolist())
```

- A. [True, True, True]
- B. ['a', 'a']
- C. [True, False, True]
- D. [False, True, False]

20. What does this print?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2], 'b': [3, 4]})
out = df.apply(lambda r: pd.Series([r['a'] + r['b'], r['a'] * r['b']]),
               axis=1)
print(out.shape)
```

- A. (2,)
- B. (1, 2)
- C. (4, 1)
- D. (2, 2)

# Answers and Explanations

Organized in the same order as the questions. Each option is explained, the correct one and the three distractors.

## pandas Answers

### 1. Correct answer, B

A. [1, 1]

Assumes pandas coerces the bool into a plain number the way it would for a numpy array, collapsing True down to 1.

B. (correct) [1, True]

Correct. Pandas keeps a bool mixed with an int as separate Python objects rather than silently coercing the bool to a number, so the original int 1 and the original True come back unchanged.

C. [True, True]

Assumes the int gets coerced up to a bool instead, turning the 1 into True.

D. [1.0, 1.0]

Assumes the mix gets promoted to a floating point dtype the way an int and a missing value would.

*A plain numpy array silently treats True as 1, but a pandas Series holding a genuine mix of int and bool falls back to the object dtype and keeps every value as the exact Python object it started as. Nothing here gets coerced to a common numeric type.*

### 2. Correct answer, A

A. (correct) ValueError

Correct. `astype(int)` tries to convert every entry, and since 'x' cannot be parsed as an integer, the conversion fails for the whole Series and raises a ValueError rather than silently skipping or defaulting that one entry.

B. nan

Assumes the whole operation quietly produces a single missing value instead of raising an error.

C. 0 1\n1 2\n2 NaN\ndtype: float64

Assumes an unparseable entry becomes a missing float value while the dtype falls back to float64, rather than the conversion raising an error.

D. 0 1\n1 2\n2 0\ndtype: int64

Assumes an unparseable entry gets silently replaced by 0 while the rest convert normally.

*astype(int) has no fallback for a value it cannot parse, it either converts every entry or raises. Unlike `pd.to_numeric with errors='coerce'`, there is no way to ask `astype` to turn unparseable entries into missing values instead of failing.*

### 3. Correct answer, A

A. (correct) [4, 5]

Correct. `s > 3` builds a boolean mask that is True only where the value exceeds 3, and indexing the Series with that mask keeps only those matching rows, 4 and 5.

B. [3, 4, 5]

Includes 3 as if the comparison were greater than or equal to 3 rather than strictly greater than.

C. [True, True]

Prints the boolean mask itself instead of the values it selects.

D. [1, 2, 3]

Keeps the rows that fail the condition instead of the ones that pass it.

*Indexing a Series with a boolean Series of the same length filters it down to the positions where the mask is True. The comparison itself is strictly greater than, so a value equal to the threshold is excluded, not kept.*

#### 4. Correct answer, A

A. (correct) [5, 8]

Correct. `isin` checks each value of `a` against the given list and returns `True` wherever it matches one of the members, so the rows where `a` is 1 or 4 pass through, giving `b` values 5 and 8.

B. [6, 7]

Keeps the rows whose `a` value is not in the list instead of the ones whose value is in it.

C. [5, 6, 7, 8]

Ignores the `isin` filter entirely and returns every row of `b`.

D. []

Assumes `isin` only matches when every value in the column equals every member of the list, so nothing passes.

*`isin` builds a boolean mask by testing membership in a collection, one value at a time, which reads more clearly than chaining several equality comparisons with `|`. It matches exactly the values listed, nothing more and nothing less.*

#### 5. Correct answer, C

A. 10

Returns the index label of the last row instead of its `a` value.

B. 2

Reads the middle row instead of the last one.

C. (correct) 3

Correct. `iloc[-1]` takes the last row by position regardless of its label, and that row has an `a` value of 3.

D. 1

Reads the first row instead of the last one.

*Negative positions with `iloc` count backward from the end exactly the way they would in a plain Python list, entirely independent of what the row labels are. The row label 30 has no bearing on which row -1 refers to.*

#### 6. Correct answer, C

A. 2.0

Drops one of the two real values along with the missing one.

B. 3

Assumes the result stays a plain integer even though the series holding it became float once a missing value was introduced.

C. (correct) 3.0

Correct. `sum()` skips missing values by default, so it adds 1 and 2, and the result prints as 3.0 because the series became float the moment `None` entered it.

D. nan

Assumes a single missing value forces the whole sum to come back missing, the way it would with `skipna` turned off.

*By default, `sum()`, `mean()` and most other reductions in pandas ignore missing values rather than propagating them. The `dtype` of the series is decided when it is built, not when it is summed, so a series that became float because of a `None` still reports its sum as a float.*

## 7. Correct answer, A

A. (correct) [1.0, nan, 3.0]

Correct. `fillna()` returns a new series by default, leaving the original one untouched, so `s` still holds its original missing value, printed as `nan` since the series is `float64`.

B. [1, None, 3]

Assumes the original series keeps its pre missing value dtype and the `None` marker rather than the float `nan` it actually holds.

C. [1.0, 0.0, 3.0]

Assumes `fillna()` changes the original series in place instead of returning a separate filled copy.

D. [1.0, 3.0]

Assumes building the filled copy also removes the missing value from the original series.

*Unless `inplace=True` is passed, `fillna()` and most other pandas methods return a new object and leave the one they were called on unchanged. The original series here was already `float64` because of the earlier missing value, so its untouched entry still prints as `nan`, not as `None`.*

## 8. Correct answer, B

A. [<NA>, <NA>, <NA>]

Assumes a single missing value anywhere in either array poisons every position of the result rather than only the ones it actually touches.

B. (correct) [2, <NA>, <NA>]

Correct. Elementwise addition of two nullable `Int64` arrays only produces a real value where both sides are present, so the first position gives 2, but the second and third each have one missing side and come back as <NA>.

C. [2, 2, <NA>]

Assumes a real value paired with a missing one on the other array still produces a real number instead of propagating the missing marker.

D. [2, 2, 3]

Assumes the missing slot on either side of the addition is simply ignored, as though it contributed nothing rather than making the result missing.

*Elementwise arithmetic between two nullable arrays follows `pd.NA` propagation position by position, a missing value on either side of a given position makes that one result missing, and it has no effect on the other positions. This is different from a whole series reduction like `sum()`, where `skipna` controls whether a single missing value anywhere poisons the entire aggregate.*

## 9. Correct answer, C

A. ['b', 'a']

Assumes the groups keep the order in which their keys first appeared in the original data.

B. [0, 1]

Assumes the index of the aggregated result is a plain positional range instead of the sorted group keys.

C. (correct) ['a', 'b']

Correct. `groupby` sorts its group keys in ascending order by default, so even though `b` appears first in the data, `a` comes first in the result.

D. ['a', 'a', 'b', 'b']

Assumes the index repeats each key once per original row instead of collapsing to one entry per group.

*The default `sort=True` makes `groupby` order its result by the group keys themselves, regardless of the order those keys first showed up in the source data. This sorting is a separate step from the aggregation and happens the same way for any aggregation function.*

**10. Correct answer, C**

A. {'a': 5, 'b': 3}

Applies the squaring and summing correctly to group a but reports group b's own plain value instead of its square.

B. {'a': 3, 'b': 3}

Sums the plain values of each group instead of squaring them first.

C. (correct) {'a': 5, 'b': 9}

Correct. The lambda squares every value in a group before summing them, so group a becomes 1 squared plus 2 squared, 5, and group b becomes 3 squared alone, 9.

D. {'a': 9, 'b': 9}

Squares the already summed total of each group instead of squaring the individual values before adding them.

*A custom function passed to agg() receives the whole group as a series and can do arbitrary computation on it before returning a single summary value. Whatever that function computes, sum of squares here, runs independently inside each group, exactly like a built in aggregation would.*

**11. Correct answer, C**

A. (3, 3)

Treats the merge as if only one pair of the duplicated key had matched instead of every combination of the two occurrences on each side.

B. (6, 3)

Multiplies as if every key, not just the duplicated one, produced several matches.

C. (correct) (4, 3)

Correct. Key 1 appears twice on each side, so an inner join produces two times two, four matching rows for that key alone, while keys 2 and 3 have no match on the other side and are dropped, giving four rows in total.

D. (2, 3)

Assumes duplicate keys line up pairwise instead of multiplying, as if row order matched row order across the two frames.

*When a key value is duplicated on both sides of a merge, every occurrence on the left is paired with every occurrence on the right, so the number of matching rows is the product of how many times the key appears on each side. A key present on only one side contributes nothing to an inner join.*

**12. Correct answer, D**

A. ['x\_x', 'y\_y']

Assumes overlapping column positions get suffixed the way a merge would, even though these two column names do not actually collide.

B. ['x', 'x']

Assumes axis=1 still stacks the frames as rows and repeats the same column name for both.

C. ['y', 'x']

Reverses the order the two frames were listed in.

D. (correct) ['x', 'y']

Correct. Concatenating along axis=1 places the frames side by side as new columns rather than stacking rows, and since the two frames have different column names, x and y, both survive unchanged.

*concat with axis=1 aligns the inputs on their index and places their columns next to each other, it does not rename or merge columns the way pd.merge does. Column name collisions are only a problem if two of the concatenated frames actually share a column label, which x and y here do not.*

**13. Correct answer, D**

A. list [1, 2, 3, 4]

Assumes the result loses its pandas Series identity and becomes a plain Python list.

B. Series [1, 2]

Assumes only the second Series survives the concatenation instead of both being stacked together.

C. DataFrame [1, 2, 3, 4]

Assumes stacking two Series with concat upgrades the result to a DataFrame the way combining them with a dict of columns would.

D. (correct) Series [1, 2, 3, 4]

Correct. Concatenating two Series with concat keeps the result as a single Series, stacking their values in order, so the type name prints as Series and the combined values are 1, 2, 3, 4.

*concat inspects what it is given and keeps the same container kind for the result, a list of Series concatenates into one longer Series, just as a list of DataFrames concatenates into one DataFrame. Both inputs sharing the same name here has no effect on this, the values are simply stacked end to end in the order the inputs were listed.*

#### 14. Correct answer, A

A. (correct) ['id', 'variable', 'value']

Correct. Without value\_vars, melt treats every column that is not listed in id\_vars as something to melt, and the output columns are always id\_vars first, then variable, then value, in that fixed order.

B. ['id', 'a', 'variable', 'value']

Assumes the original a column stays in the frame alongside the new variable and value columns.

C. ['variable', 'value']

Drops the id column entirely, as if id\_vars only affected which rows are kept.

D. ['id', 'value', 'variable']

Swaps the order of the variable and value columns.

*When value\_vars is omitted, melt melts every column not named in id\_vars, which is the same result as listing them all explicitly. Regardless of how many columns get melted, the output always has exactly the id columns followed by variable and then value, in that order.*

#### 15. Correct answer, C

A. (1, 3)

Assumes sorting collapses rows that tie on the sort key into a single row.

B. (3, 2)

Assumes sorting drops the column that was sorted on, as if it were consumed by the operation.

C. (correct) (3, 3)

Correct. sort\_values only reorders the existing rows, it never adds or removes rows or columns, so a three row, three column frame stays three by three after sorting.

D. (2, 3)

Confuses the row and column counts, as though sorting transposed the frame.

*sort\_values is a pure reordering operation on the existing rows, it changes the sequence in which rows appear but never their number, and it never touches the set of columns at all. Any operation that would change the shape of a frame, like dropping duplicates or aggregating, is a separate step from sorting.*

#### 16. Correct answer, C

A. ChainedAssignmentError

Assumes this recommended single step pattern still triggers the warning that chained assignment triggers.

B. [7, 8, 9]

Assumes a loc based write is chained the same way `df[mask][col]` is, and gets silently dropped.

**C. (correct) [7, 0, 0]**

Correct. `df.loc[mask, 'b'] = 0` addresses both the rows and the column in one call on `df` itself, so it is a direct write that succeeds, giving `[7, 0, 0]`.

**D. [0, 0, 0]**

Assumes the write reaches every row of the column instead of only the rows selected by the mask.

*The documentation for chained assignment recommends exactly this pattern, `.loc[row_indexer, col_indexer] = value`, because it resolves the row selection and the column selection in one call rather than two separate bracket lookups. It writes straight into `df` with no intermediate object involved.*

## 17. Correct answer, A

**A. (correct) None [1, 2]**

Correct. `drop_duplicates` with `inplace=True` removes the repeated row directly from `df` and returns `None`, so `resultat` is `None` and `df['a']` now reads `[1, 2]`.

**B. None [1, 1, 2]**

Assumes the `inplace` call returns `None` correctly but the duplicate row was never actually removed from `df`.

**C. a\n0 1\n2 2 [1, 2]**

Assumes `resultat` holds the whole deduplicated `DataFrame` rather than `None`.

**D. [1, 2] [1, 2]**

Assumes `resultat` holds the deduplicated column as a list instead of `None`.

*`drop_duplicates` with `inplace=True` follows the same convention as the other `inplace` methods here, mutate and return `None`. The duplicate is genuinely gone from `df`, the empty return value only reflects the call itself, not the state of `df`.*

## 18. Correct answer, C

**A. [10, 10, 10]**

Assumes every entry of the column ends up with the same value instead of each one being multiplied by ten individually.

**B. None**

Confuses the effect of the assignment on `df` with the return value of the statement itself, which prints nothing at all.

**C. (correct) [10, 20, 30]**

Correct. `df.loc[:, 'a'] = ...` is a single step write directly on `df`, so the whole column is replaced with its values multiplied by ten, giving `[10, 20, 30]`.

**D. [1, 2, 3]**

Assumes this loc based write is somehow chained and gets silently dropped the way `df[mask][col]` would be.

*Using `.loc` with both a row selector and a column label on the left of an assignment addresses `df` directly in one call, exactly like the recommended fix for chained assignment. Reading `df['a']` on the right hand side happens first and does not itself create any lasting alias, so the multiplication result is written straight into `df`'s own column.*

## 19. Correct answer, C

**A. [True, True, True]**

Assumes `startswith` checks for the letter anywhere in the string instead of only at its very first position.

**B. ['a', 'a']**

Assumes `startswith` extracts the matching prefix itself instead of reporting a boolean for each row.

**C. (correct) [True, False, True]**

Correct. `str.startswith` checks only the first character of each string against 'a', which apple and avocado satisfy and banana does not.

D. [False, True, False]

Reports the exact opposite of each row, as if the check were for `endswith` or a negated condition.

*str.startswith mirrors the built in str.startswith method, testing only the beginning of each string, and it returns one boolean per row rather than extracting any part of the text.*

## 20. Correct answer, D

A. (2,)

Assumes returning a Series from the row function still leaves the overall result as a single column Series.

B. (1, 2)

Swaps rows and columns, treating the two original rows as if they became two output columns instead of two output rows.

C. (4, 1)

Assumes the two values produced per row get stacked into one long column instead of laid out as two columns.

D. (correct) (2, 2)

Correct. When the function passed to `apply` with `axis=1` returns a Series, pandas lays its values out as columns of a new DataFrame, one row per original row and one column per element of the returned Series, giving a two by two shape.

*apply inspects what the row function returns, a scalar produces a plain Series result, but a Series return value gets expanded into a full DataFrame, with the returned index becoming the new columns.*

## **That was 20 of the 255 pandas questions.**

The full bank is one PDF laid out exactly like this sample, all 255 questions first and the complete answer key after, with a reason for each of the four options. Every snippet in it was run the same way before its answer was written.

Get it for 9 euros at <https://ko-fi.com/s/4b32c5be4f>, or SQL, Python, pandas and NumPy in one PDF for 19 euros at <https://ko-fi.com/s/8151d45764>.

The same 20 questions can be played in a browser at <https://thibaudlepan77-svg.github.io/interview-questions-verified/>. If an answer looks wrong to you, open an issue on the repository linked from that page with the question number and what your own run printed.